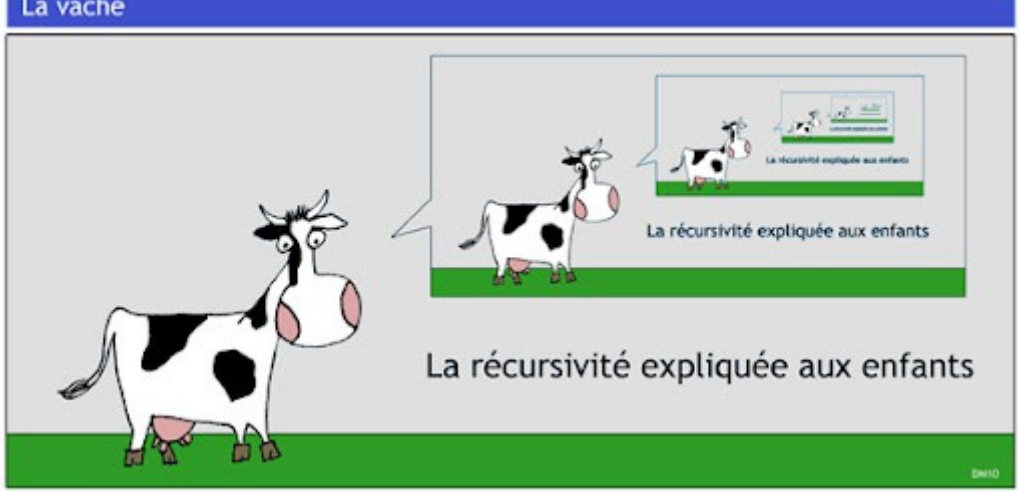
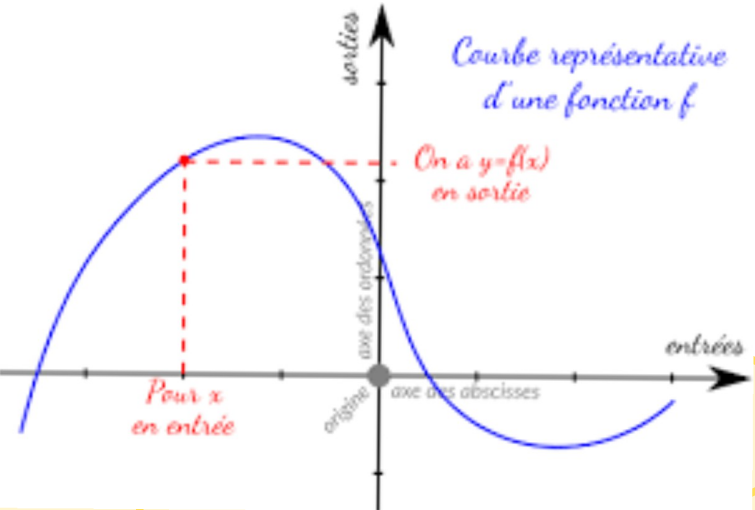


Python : programmation web



Cours 7 : La récursivité et les moteurs de recherche

```python
def is_palindrome(substring):
    if len(substring) < 2:
        return True

    for i in range(len(substring) // 2):
        if substring[i] == substring[-(i+1)]:
            return True

    return False

def longestPalindrome(s):
    longest = 0
    longest_substring = ""

    for i in range(len(s) + 1):
        for j in range(i, len(s) + 1):
            substring = s[i:j]

            if is_palindrome(substring):
                if len(substring) > longest:
                    longest = len(substring)
                    longest_substring = substring

    print(longest_substring)
    return longest_substring
```

i	j	substring	Ce que fait is_palindrome(substring)	Résultat	longest	longest_sub.
0	0	""	len < 2 → renvoie True (mais 0 > 0 est faux)	True	0	""
0	1	"c"	len < 2 → renvoie True (1 > 0 : **mise à jour**)	True	**1**	"c"
0	2	"ca"	1er tour : 'c' == 'a' est faux → fin boucle	False	1	"c"
0	3	"car"	1er tour : 'c' == 'r' est faux → fin boucle	False	1	"c"
0	4	"cara"	1er tour : 'c' != 'a' ; 2e tour : 'a' != 'r'	False	1	"c"
0	5	"carac"	1er tour : 'c' == 'c' est vrai → **return True (mise à jour)**	True	**5**	"carac"
0	6	"caraca"	1er tour : 'c'!='a' ; 2e : 'a'!='c' ; 3e : 'r'!='a'	False	5	"carac"
0	7	"caracas"	1er tour : 'c' != 's' (continue!) ; **2e tour : 'a' == 'a' → True !**	True	**7**	"caracas"
1	1	""	len < 2 → True (mais 0 > 7 est faux)	True	7	"caracas"
1	2	"a"	len < 2 → True (mais 1 > 7 est faux)	True	7	"caracas"
1	3	"ar"	1er tour : 'a' != 'r' → fin boucle	False	7	"caracas"
1	4	"ara"	1er tour : 'a' == 'a' → True (vrai palindrome, mais 3 <= 7)	True	7	"caracas"
1	5	"arac"	1er tour : 'a' != 'c' ; 2e tour : 'r' != 'a'	False	7	"caracas"
1	6	"araca"	1er tour : 'a' == 'a' → **return True à tort !** (mais 5 <= 7)	True	7	"caracas"
1	7	"aracas"	1er tour : 'a'!='s' ; 2e : 'r'!='a' ; 3e : 'a'!='c'	False	7	"caracas"

L'erreur se trouve dans la fonction `is_palindrome` aux lignes 5 à 9 :

```
    for i in range(len(substring) // 2):
        if substring[i] == substring[-(i+1)]:
            return True
    return False
```

Écrite comme cela, la fonction renvoie `True` dès qu'elle trouve **une seule** paire de lettres symétriques identiques dans le mot. Or, pour être un palindrome, il faut que **toutes** les paires symétriques soient identiques.
Il faut donc faire l'inverse : dès que deux lettres symétriques sont **différentes** (`!=`), on peut s'arrêter et renvoyer `False`. Et ce n'est qu'à la toute fin de la boucle (si aucune différence n'a été trouvée) qu'on renvoie `True` :

```
def is_palindrome(substring):
    if len(substring) < 2:
        return True

    for i in range(len(substring) // 2):
        if substring[i] != substring[-(i+1)]:
            return False

    return True
```

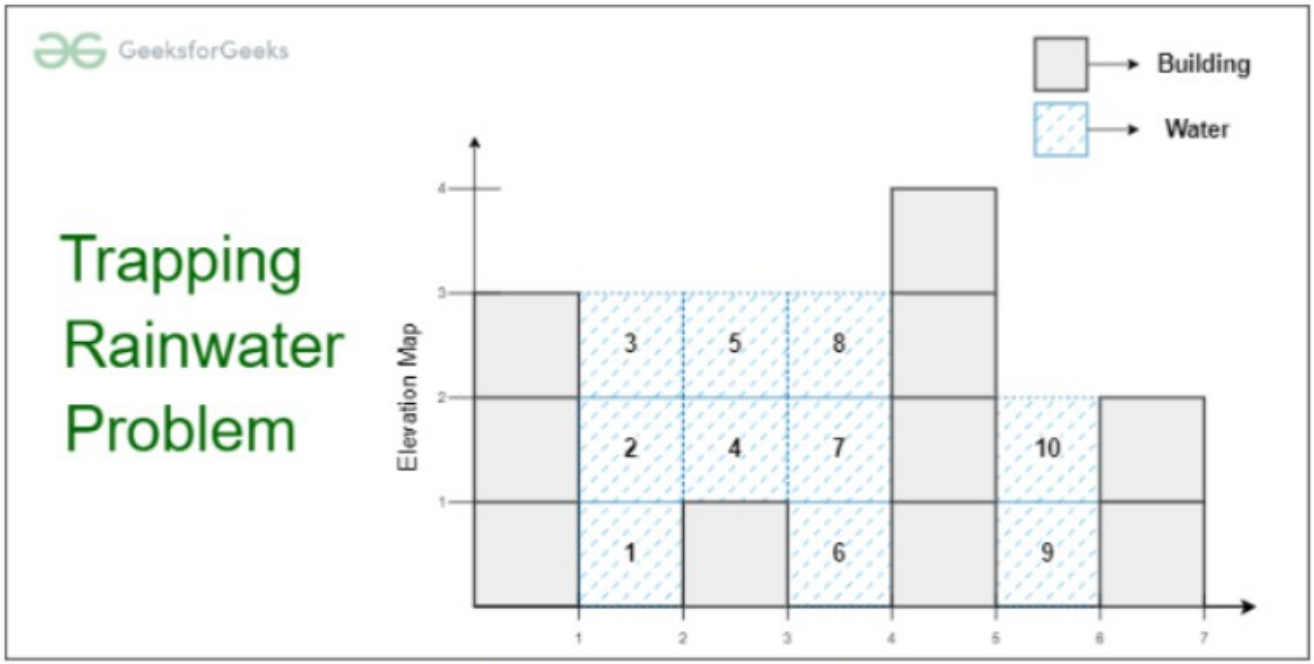
Remarque : en Python, on peut aussi écrire directement ***`return substring == substring[::-1]`***.
Une fois cette correction faite, `longestPalindrome("caracas")` affiche et renvoie bien `"carac"`.

Exercice sur les pythonismes : Python gym



Interview 6 : piéger l’eau de pluie

Lisez la description du problème sur le site https://leetcode.com/problems/trapping-rain-water/description/



Problème de rétention des eaux de pluie

1) comprendre le problème

```
def trap( height):
    """
    :type height: List[int]
    :rtype: int
    """

    def compute_water_blocks_on_level_n(n) :
        rock_index = []
        for i, level in enumerate(height) :
            if level > n :
                rock_index.append(i)
        return rock_index[-1] +1 - rock_index[0] - len(rock_index)

    total = 0
    max_level = max(height)

    for i in range(max_level) :
        total += compute_water_blocks_on_level_n(i)

    return total
```

2) parcours vertical

```python
def trap( height):

        # On part de chaque côté et on réduit
        left, right = 0, len(height) - 1
        left_max, right_max = height[left], height[right]
        total = 0


        while left < right:
            if left_max < right_max:
                left += 1
                left_max = max(left_max, height[left])
                total += left_max - height[left]

            else:
                right -= 1
                right_max = max(right_max, height[right])
                total += right_max - height[right]

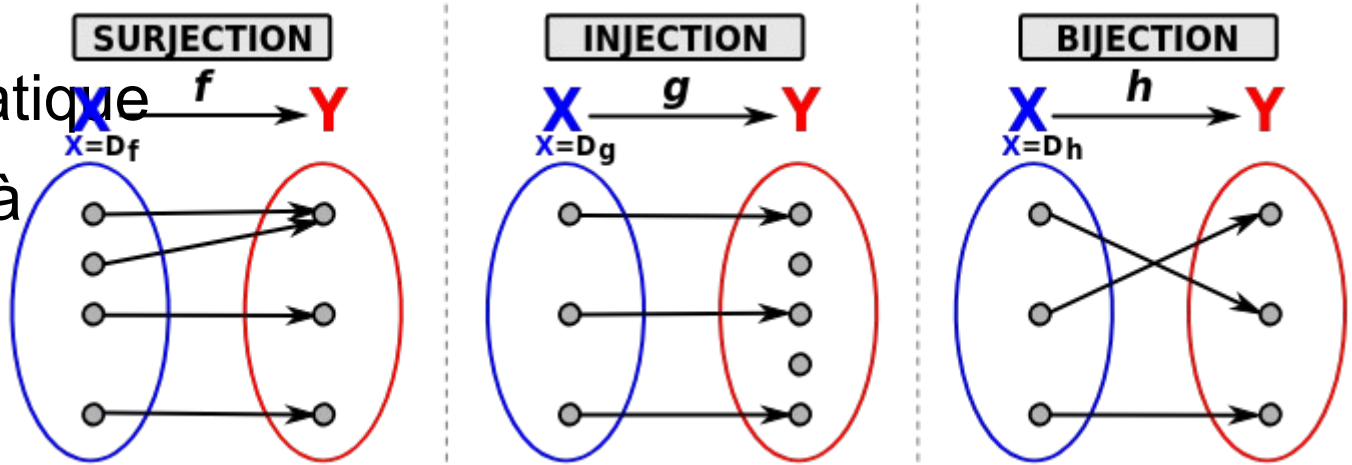
        return total
```

Python : les aspects fonctionnels

1) Qu’est ce qu’une fonction ? Son application dans python
2) comment utiliser python comme un langage fonctionnel
3) la récursivité : définition usage et exemples

1) Qu'est ce qu'une fonction

- Une fonction désigne avant tout un *mapping*, une association entre une entrée et une sortie (chacun pouvant être composé de plusieurs éléments)
- Domaine de définition
- Une fonction mathématique associe chaque input à la même output pas en informatique (random)



1) qu’est ce qu’une fonction

En informatique, la définition est plus générale : désigne un processus qui transforme une entrée en une sortie.

- De façon plus pragmatique désigne un bloc de code qui prend en charge une ou plusieurs tâches spécifiques.

  Une fonction informatique peut : ne rien renvoyer et modifier , modifier l’état interne d’un objet, interagir avec l’environnement… (def pass() : pass est une fonction de plein droit) vs « fonciton pure »
- Subroutines Procedures Methods Subprograms
- => manière de factoriser des tâches qu’on a à répéter

1) qu’est ce qu’une fonction

def <function_name>([<parameters>]):

    <statement(s)>

required/posotional arguments vs keywords /optional arguments

arguments par défaut : attention à ne pas donner d’arguments pra défauts mutables !!! (list = [] … NON il vaut mieux mettre list = None et if list is None : list = []

1) qu’est ce qu’une fonction

- Les variables au sein des fonctions :
- Python crée un nouvel espace de référence
- Dans python tout est objet donc les réassignations ne sont que des réassignations de pointeurs
- Dédoublement entre la variable donnée en entrée et la variable nécessaire au sein de la fonction
- Pass-by-assignment / pass-by-object
- Ne vaut pas pour la modofication d’objets (lists, dics) car aucun réassignement n’a lieu

1) qu’est ce qu’une fonction

Structure d’une fonction :

commence par sa définition, indentation

Sortir de la fonction : return optionnel, mécanisme de retour d’appel et sort de l’exécution du bloc (évaluation de l’appel à la fonction)

l’appel renvoie None si rien de spécifié ou pas de return

1) qu'est ce qu'une fonction

- En python une fonction est avant tout un objet qui implémente la fonction __callable_() : que l'on peut appeler avec son nom suivi de parenthèses

  (d'autres objets comme les classes implémentent aussi)

- On peut définir le __call__ de certaines classes pour :

  *créer des comportements spéciaux pour ces classes (side effect)

  * retenir l état entre plusieurs appels (caching) stateful callables.

  * créer des classes dont les instances sont des fonctions

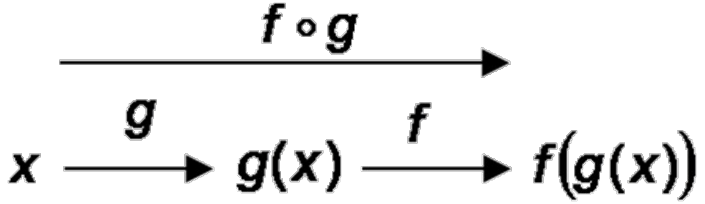
2) La programmation fonctionnelle en python

La programmation fonctionnelle est un paradigme de programmation selon lequel le code s'organise autour de l'exécution de fonctions.

Les fonctions se composent entre elles pour créer des fonctions plus complexes.

- Dans le paradigme fonctionnel on utilise que des « fonctions pures » qui ne modifient pas l'état

  interne d'éléments mutables



2) La programmation fonctionnelle en python

- Une fonction en python est un objet comme les autres

  ex len est un objet fonction , len(x) l'exécution. On peut assigner une fonction comme valeur d'une variable

2) La programmation fonctionnelle en python

Point clé : une fonction peut prendre en entrée une fonction et retourner une autre fonction

terme « call back » function celle donnée comme paramètre

- Car la fonction donnée en paramètre modifie le comportement de la fonction exécutée

  ex : sorted(liste_de_string, key=len)

2) La programmation fonctionnelle en python

- Parfois on définit des fonctions anonymes : les fonctions lambda !

  Utile surtout pour des petites fonctions faciles qui servent de callback function

  lambda <parameter_list>: <expression>

  Cela définit un objet python qui est une fonction comme les autres
- (lambda s: s[::-1])("Je retourne le str")

  print_last = lambda : liste : print(liste[-1])

  équivalent de

  def print_last(liste) :

  print(liste[-1])

2) La programmation fonctionnelle en python

Trois fonctions principales dans python implémentent la façon de penser fonctionnelle :

map()

filter()

reduce()

2) La programmation fonctionnelle en python

Map() : permet d'appliquer une fonction à chaque élément d'un itérable , renvoie un itérateur qui renvoie le résultat

- map(<f>, <iterable>)
- map(<f>, <iterable$_1$>, <iterable$_2$>, ..., <iterable$_n$>) un itérable par argument de la fonction !

sorte d'equivalent de

```
for i in iterable :
    return f(i)
```

2) La programmation fonctionnelle en python

- filter(<f>, <iterable>) applique le filtre sur les éléments de l'itérable et ne renvoie que les éléments pour lesquels l'application de f renvoie True

2) La programmation fonctionnelle en python

- Reduce : réduit un itérable à une valeur unique
- reduce(<f>, <iterable>, <initializer>)

  utile pour les cas où on effectue des opérations sur l'ensemble des valeurs (cf pandas : apply vs some operations)

  la fonction doit prendre en entrée 2 arguments et renvoie l'application à la chaine de la fonction avec l'élément i+1 avec le résultat de l'application de [:i]
- ex : calcul de moyenne

  Ne fait plus partie du langage de base car le créateur de python

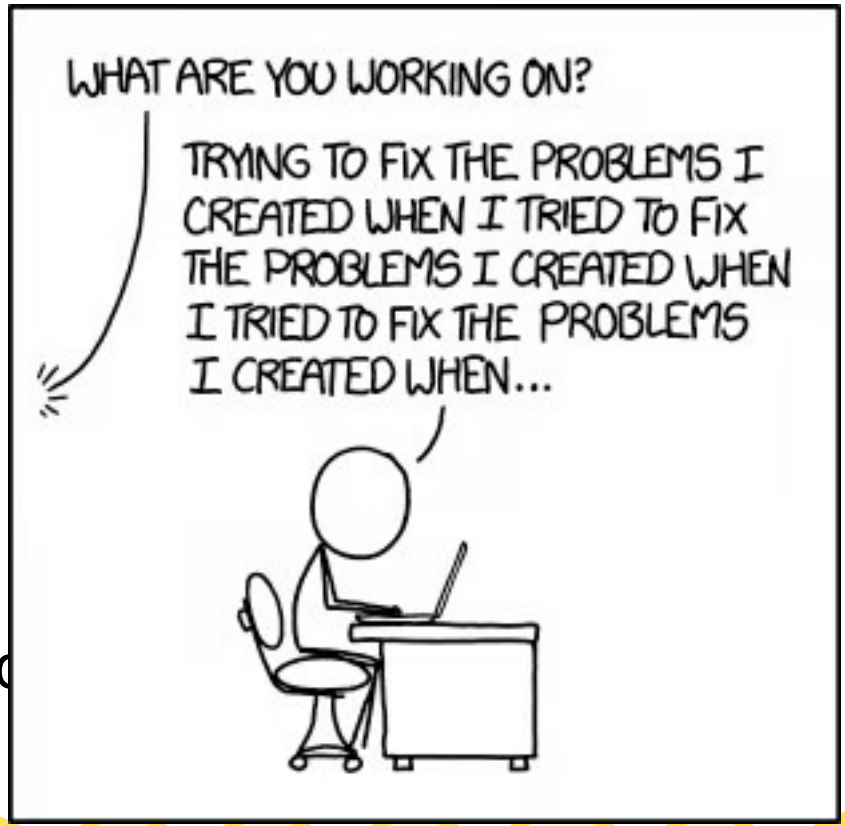
3) Récursion

Quand une fonction est appelée dans sa propre définition

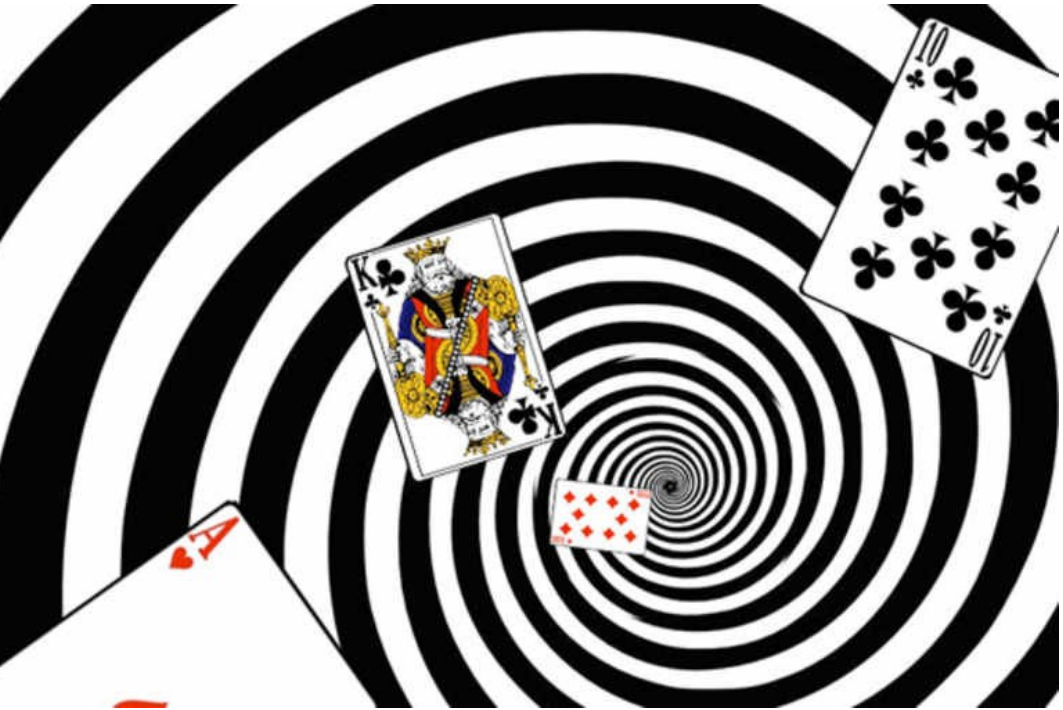
utile quand on définit des objets comme des suites numériques

les traversées de structures de c complexes comme les arbres...

notion d'auto référencement



3) Récursion



- MAIS
- Cette approche n’est pas adaptée à tous les problème
- Souvent a une complexité plus grande
- Il faut s’assurer que le programme termine !

  RecursionError: maximum recursion depth exceeded

```python
def appeler_eleves(eleves):  2 usages
    # Cas de base : si la liste est vide, on arrête l'appel
    if not eleves:
        print("Tous les élèves ont été appelés.")
        return

    # Appel du premier élève
    print(f"Présent : {eleves[0]}")

    # Appel récursif pour les autres élèves
    appeler_eleves(eleves[1:])


# Exemple d'utilisation avec 10 élèves
liste_eleves = ["Alice", "Bob", "Clara", "David", "Emma", "Farid", "Gabriel", "Hélène", "Ismaël", "Jade"]
appeler_eleves(liste_eleves)
```

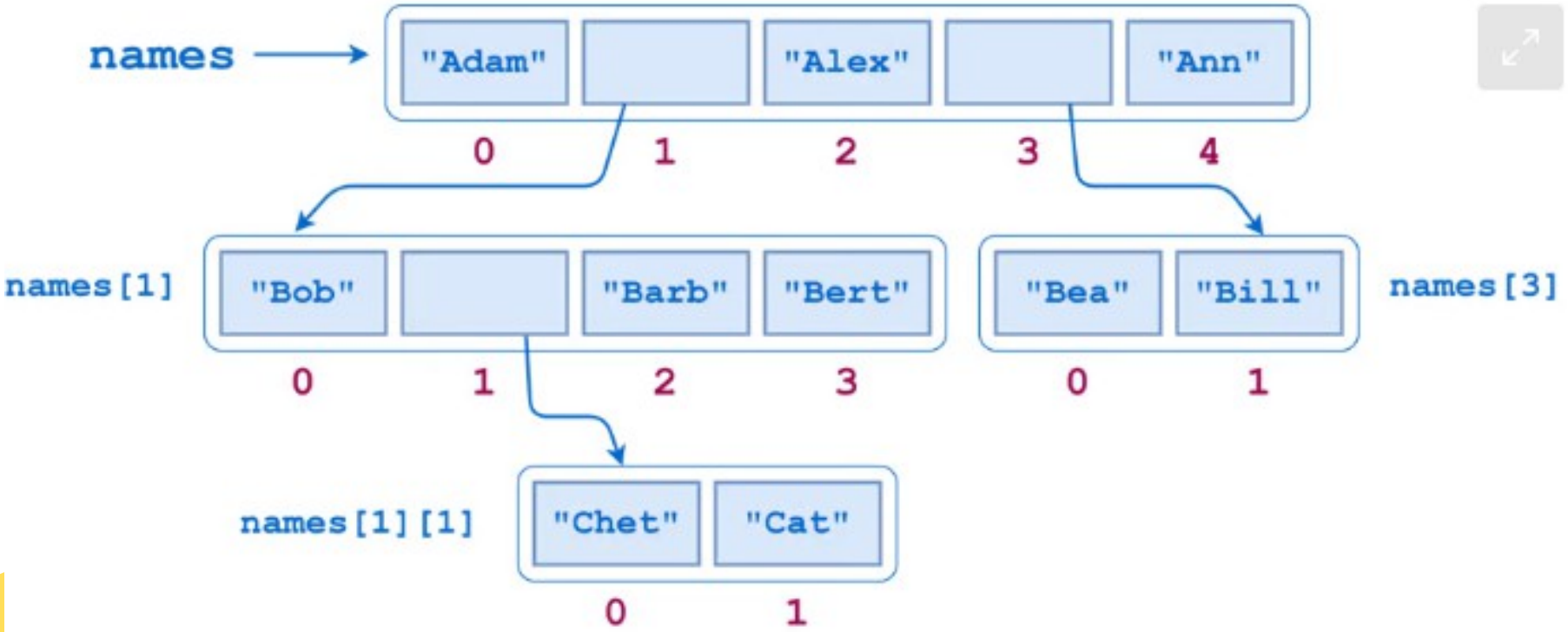
L’exemple classique : la factorielle

```
>>> def factorial(n):
...     print(f"factorial() called with n = {n}")
...     return_value = 1 if n <= 1 else n * factorial(n -1)
...     print(f"-> factorial({n}) returns {return_value}")
...     return return_value
...

>>> factorial(4)
factorial() called with n = 4
factorial() called with n = 3
factorial() called with n = 2
factorial() called with n = 1
-> factorial(1) returns 1
-> factorial(2) returns 2
-> factorial(3) returns 6
-> factorial(4) returns 24
24
```

Utile pour parcourir des structures de données en profondeur

- Source : realpython



3) Récursion

Quelques stratégies/patterns à avoir en tête

- Reductive recursion : ce qui est envoyé dans les appels internes est plus court / rapide à traiter que l'input de départ
- Counter. Avoir une variable de contrôle qui stocke les informations

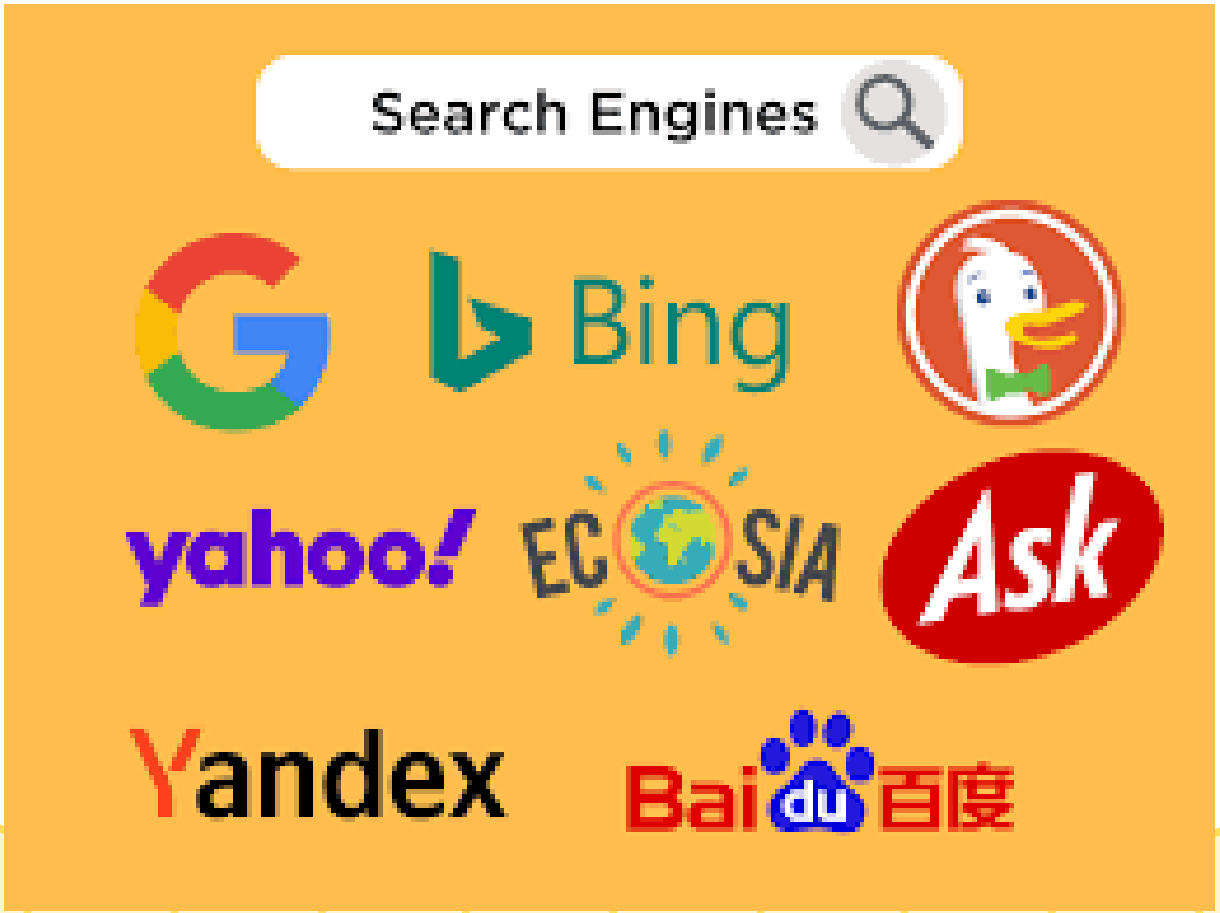
  il faut pouvoir avoir un oeuil sur l'état global du traitement de la fonction
- Système de cache pour optimiser le nombre d'appel interne

references

- https://realpython.com/defining-your-own-python-function/
- https://realpython.com/python-namespaces-scope/#namespaces-in-
- https://realpython.com/python-recursion/
- https://realpython.com/python-callable-instances/

  Oui je sais, j'aime bien ce site…

Les moteurs de recherche



Qu’est-ce qu’un moteur de recherche (search engine)



https://lasueur.com/batman-paul-dano-homme-mystere-18-11-2019

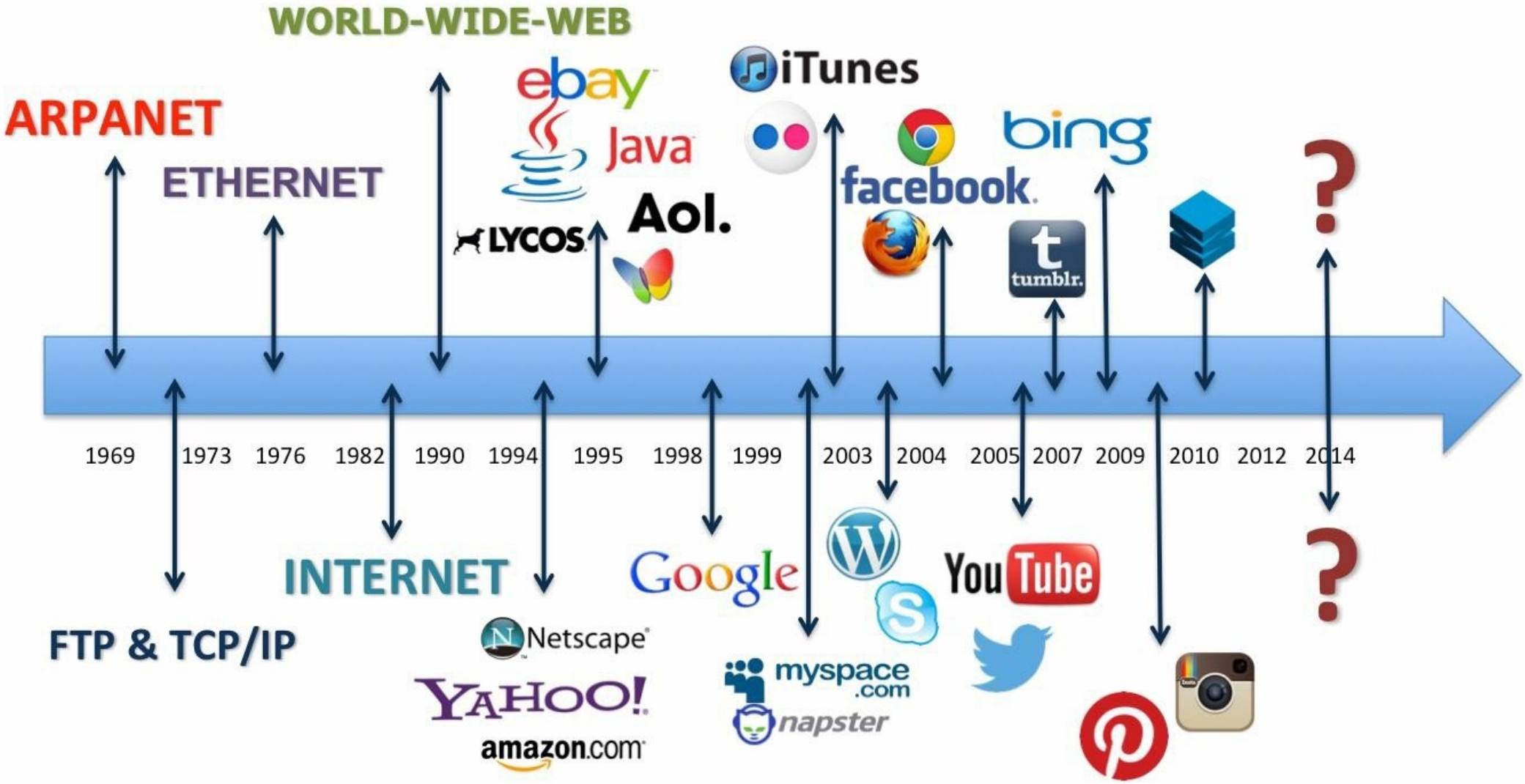
Qu’est-ce qu’un moteur de recherche (search engine)

* Plus les capacités des ordinateurs ont augmenté, plus les gens ont mis des choses dedans (fichiers, données)

* Et plus c’était difficile de s’y retrouver.

* Le besoin s’est encore plus affirmé avec internet lorsqu’on a voulu chercher des choses dans les ordinateurs des autres (fin les serveurs)

=> Un moteur de recherche est un programme dont le but est de fournir à l’utilisateur des informations correspodant à sa demande



https://www.seomechanic.com/complete-history-search-engines/

1er moteur de recherche Archie

- 1990
- Permet de filtrer parmi des fichiers qu'on a téléchargés

Le problème : chercher quoi et où ?

Cela a commencé en local à partir de données web

Milestone : 1994 Yahoo

- 1994– David Filo and Jerry Yang créent Yahoo! Search.
  Première collection des URL de pages web jamais créée
- Contiennent des descriptions rédigées à la main des pages
-
- WebCrawler–
- Même année
- Premier moteur qui indexe le texte de toute la page web

1996-8 L’effet Google

- Larry Page and Sergey Brin
- Idée que pour mieux classer les pages on peut utiliser les liens qui les relient entre elles
- C’est la pertinence supérieure de ce moteur qui en a fait le leader du marché

2010

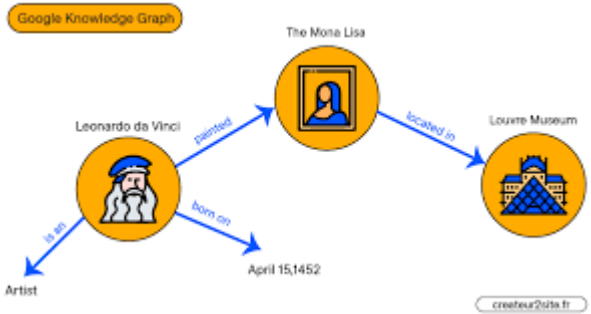
- Des index prés remplis aux index dynamiquement créés à la volée à partie de la requête utilisateur.
- 2013 hummingbird : premier algorithme qui tente de capturer « l'intetion de l'utilisateur » de façon plus fine
- Défis : lutter contre les tricheurs qui mettent des mots clés dans les pages pour augmenter leur rang.

2015-25 : la nouvelle aire du search

- 2018 Googleintroduit BERT pour encoder la query de l'utilisateur. Intégration de la recherche vectorielle

- 2012 Google Knowledge graph

  utilisation progressivement étendue

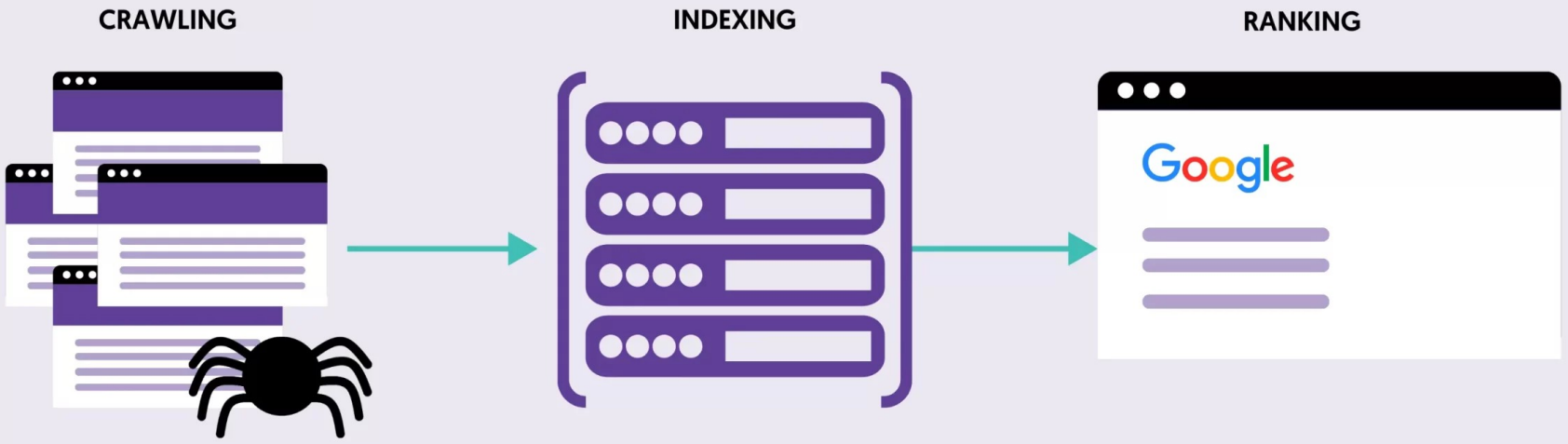


Le futur du search ?



perplexity

How Search Engines Work



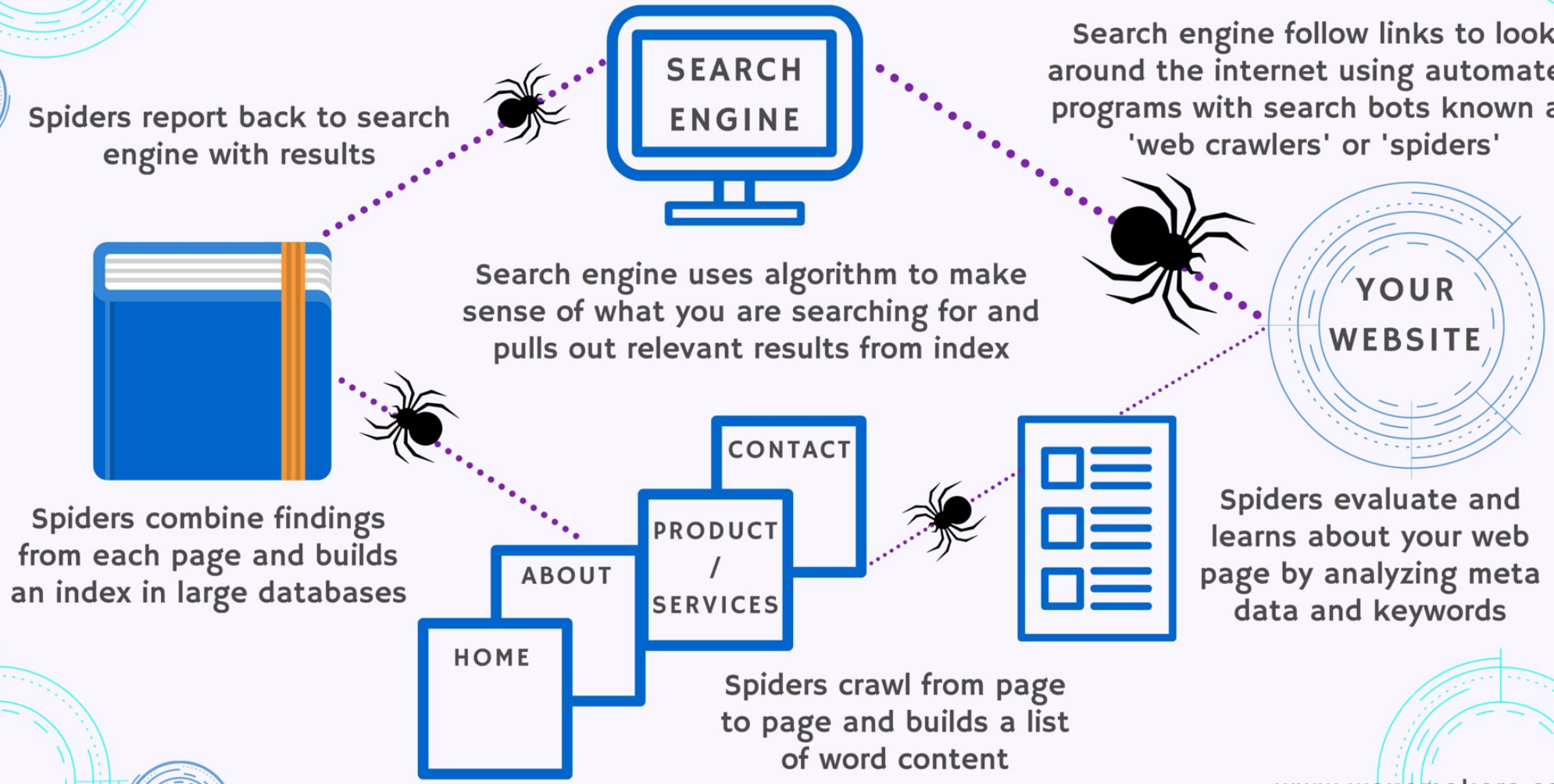
Google bots crawl the web to find new pages or updated content.

Information identified by the bots is organized, sorted and stored in Google Index.

Google ranking algorithms decide which pages to show in the SERPS and in what order.

https://www.google.com/imgres?q=how%20do%20search%20engine%20work%20%3F&imgurl=https%3A%2F%2Fwww.reliablesoft.net%2Fwp-content%2Fuploads

HOW DO SEARCH ENGINES WORK?



0) Crawl : cf cours 3
1) indexation

On extrait tout le contenu de la page

On le range dans un index

On crée pour chaque mot clé la liste des pages où il apparaît avec des poids

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF - IDF(t,d) = TF(t,d) * IDF(t)$$

2) ranking

À partir de la query de l'utilisateur on extrait les mots clés et on fait une formule.

- NLP old school : on fait une somme pondérée du poids de chaque mot dans la requête utilisateur par rapport aux mots contenus dans la page (index, reverse search)

  Cf BM25

- Pondération par l'importance et la qualité relative de chaque page (Page Rank : LA raison du succès de google)

On calcule le poids d'une page à partir de la formule mathématique suivante :

$$PR(p_i) = (1 - d) + d \cdot \sum_{p_j \in L(p_i)} \frac{PR(p_j)}{C(p_j)}$$

PR(pi) : Le score PageRank de la page pi. C'est la mesure de l'importance de la page pi dans le graphe et que l'on souhaite calculer.

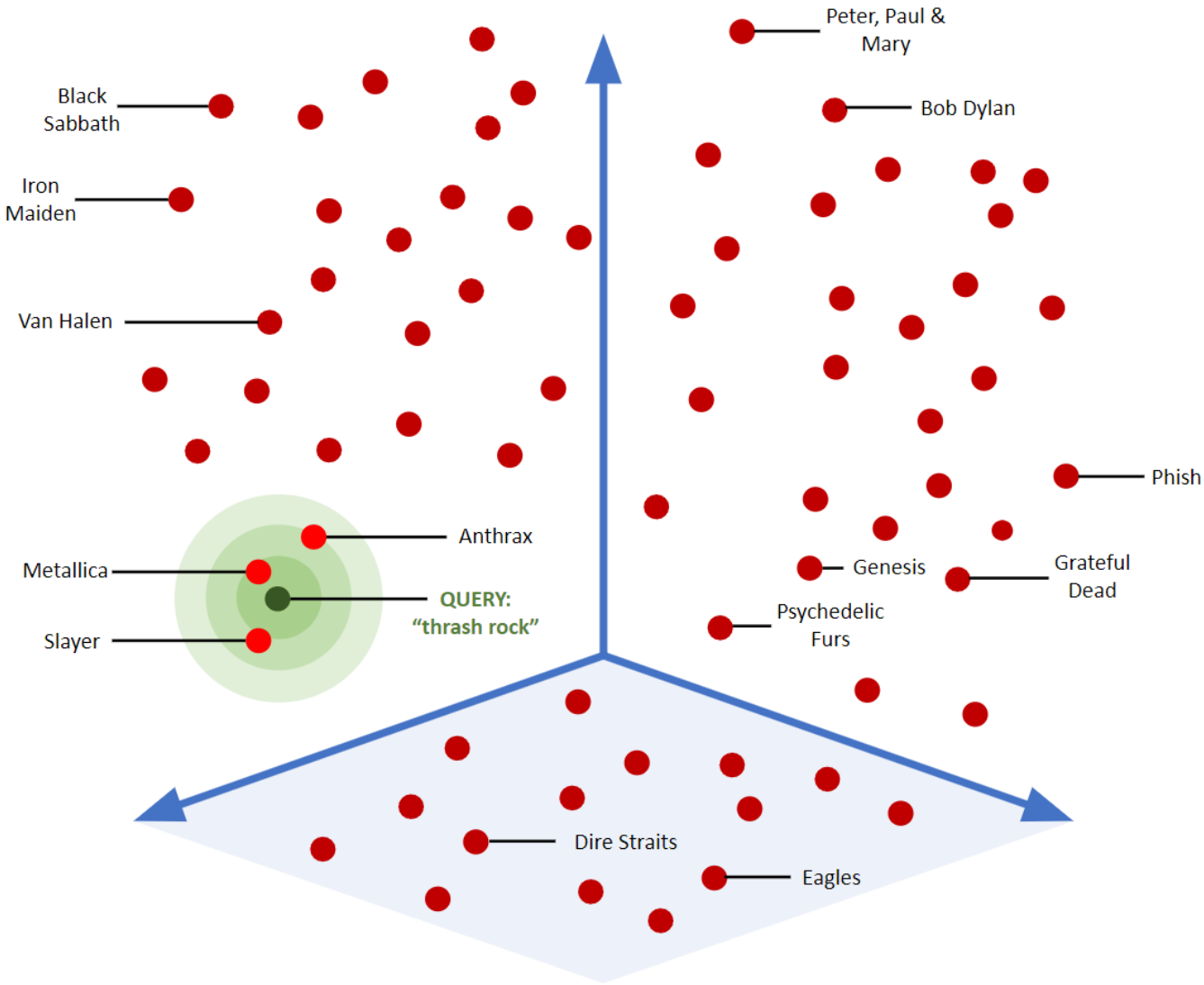
d : Le facteur de **damping** (ou d'atténuation), généralement choisi entre 0.85 et 0.90. Il modélise la probabilité qu'un utilisateur continue de suivre des liens sur une page ou décide de visiter une autre page de manière aléatoire.
L(pi) : L'ensemble des pages pj qui contiennent un lien vers pi (les pages qui pointent vers pi).

PR(pj) : Le score PageRank de la page pj, qui pointe vers pi.
C(pj) : Le nombre total de liens sortants depuis la page pj. Le PageRank est divisé également entre toutes les pages vers lesquelles pj pointe (une page qui cite tout et n'importe quoi donne moins d'importance au fait de renvoyer vers un lien).

https://www.youtube.com/watch?v=u8HtO7Gd5q0

Leveraging vector search

- Ou reranking



3) Autre tricks du search

- Optimiser les requêtes (query expansion)

  = transformer la requête de l utilisateur en une requête optimisée pour le moteur

  https://hal.science/hal-05046658v1

- Les coulisses de l'indexation

  comment la page est transformée dans l'index, quelles métadonnées sont stockées, pour fournir le classement

  (C'est une recette secrète!!!)